

# Transformer Scaling Exercises

July 13, 2026

## 1 Introduction

I summarize my derivation of Chinchilla-style scaling law parameters for dense and Mixture-of-Experts (MoE) transformer architectures on the task of simple 3 digit addition, and the design of a custom Pallas TPU kernel that beats `jax.lax.ragged_dot` by fusing the up/down projection for an MoE.

## 2 Chinchilla Laws

### 2.1 Experimental Setup

For dense models, parameters ranged from approximately 400K to 21M, with training dataset sizes ranging from 10K to 2M tokens. MoE models used 8 experts with top- $k = 2$  routing, ranging from approximately total parameters 1.3M to 80M (activated parameters ranging from approximately 600K to 37M), trained on the same token range.

### 2.2 IsoFLOP Curves

As per Approach 2 from the Chinchilla Laws paper, I plot IsoFLOP curves for dense and MoE model architectures. In general, both models scale best with data, with loss dropping significantly as token counts increase. I hypothesize this is because we are in the finite data regime: model's can theoretically achieve zero loss by memorizing the dataset. For dense architectures, the curves flatten out, implying that scaling model size provides minimal improvement to loss. In comparison, MoE architectures have sharper curves and show some curvature. This indicates that for this task, MoEs scale more efficiently than dense architectures with respect to parameter count.

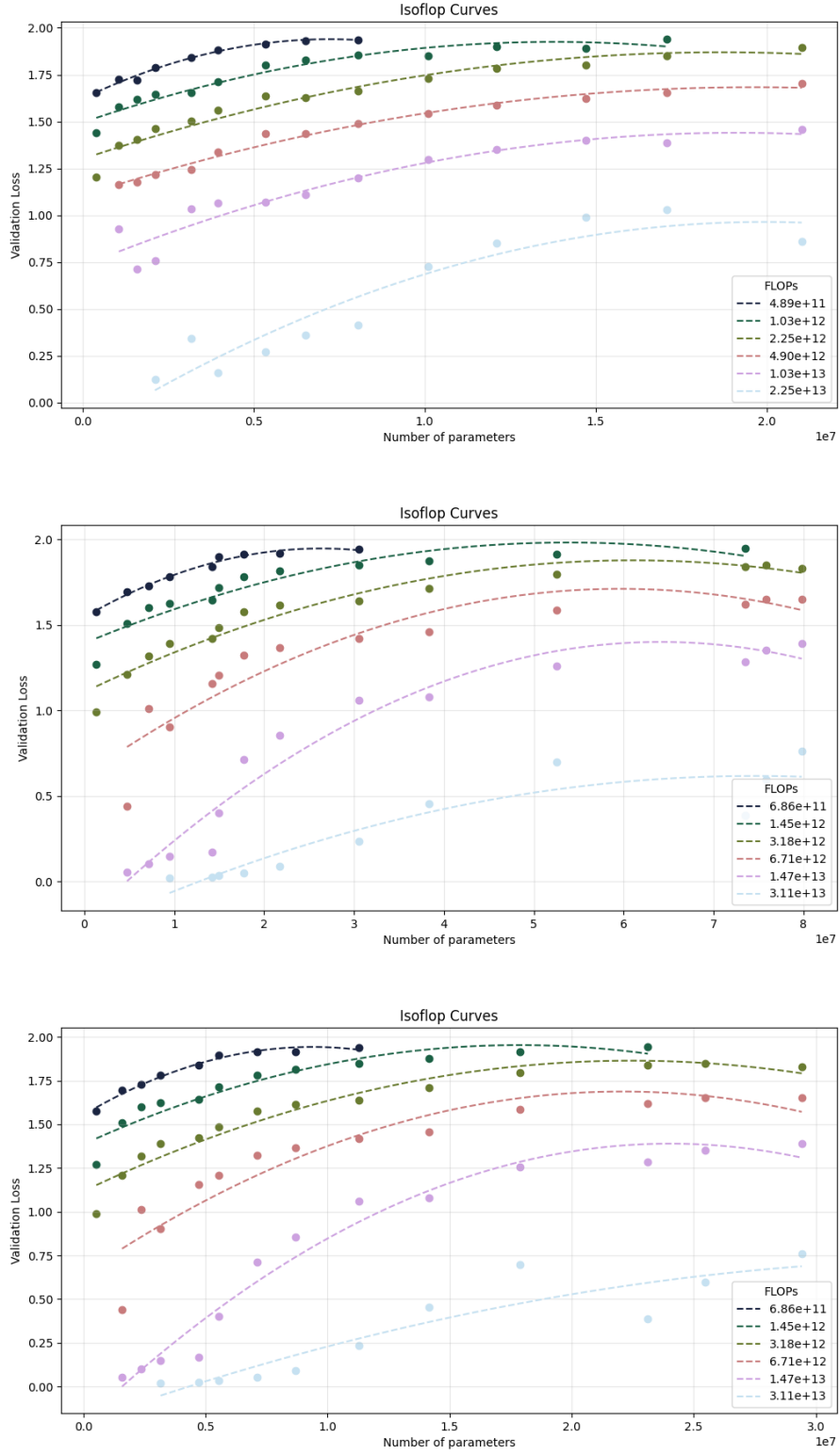


Figure 1: IsoFLOP curves for dense (left), MoE total parameters (center), and MoE activated parameters (right). Each curve corresponds to a fixed FLOP budget; the  $x$ -axis shows model size and the  $y$ -axis shows validation loss.

### 2.3 Parametric Fit

I also fit a parametric equation as per Approach 3, and plot the efficient frontier against the empirical data. The fitted loss function takes the form:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (1)$$

where  $N$  is the number of model parameters,  $D$  is the number of training tokens,  $E$  is the irreducible loss floor, and  $A$ ,  $B$ ,  $\alpha$ ,  $\beta$  are fitted constants.

Table 1: Fitted Chinchilla scaling law parameters for Equation 1.

| Parameter | MoE (Total) | MoE (Activated)     | Dense  |
|-----------|-------------|---------------------|--------|
| $A$       | 0.6347      | $3.809 \times 10^8$ | 9020.7 |
| $B$       | 153.9       | 794.9               | 512.4  |
| $E$       | 0.03370     | 0.000               | 0.000  |
| $\alpha$  | 7.792       | 3.808               | 0.7221 |
| $\beta$   | 0.6576      | 0.7997              | 0.6569 |
| $a$       | 0.07782     | 0.1736              | 0.4764 |
| $b$       | 0.9222      | 0.8264              | 0.5236 |

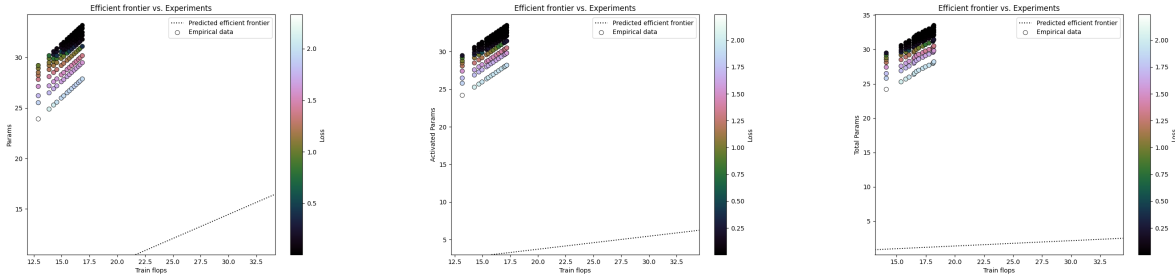


Figure 2: Efficient frontier plots for dense (left), MoE activated parameters (center), and MoE total parameters (right). Each plot compares the parametric fit from Equation 1 against empirical data points. The parametric fit predicts that the optimal allocation is uses far more data per parameter than used in my experiments.

For the dense model,  $\alpha \approx 0.72$  and  $\beta \approx 0.66$ , indicating that loss decreases at roughly comparable rates as both parameters and data are scaled, with a slight advantage to data. Notably, this is in contradiction with the dense IsoFLOPs curves, which indicates that increasing data is far more efficient than increasing parameters. I hypothesize this discrepancy is caused by a violation of the assumptions that the parametric function makes. In particular, because we are in the finite data regime, the statement "the final term captures the fact that the transformer is not trained to convergence, as we only make a finite number of optimization steps, on a sample of the dataset distribution" is violated. Additionally, I did not collect empirical data at extremely small parameter counts, which (according to the IsoFLOPs curves) is where the optimal allocation of compute for a given FLOPs budget is.

When fit on total parameters,  $\alpha \approx 7.79$  and  $\beta \approx 0.66$ , indicating that MOEs scale much harder with data than dense models. Notably  $A \approx 0.63$ , which is very small, but  $\alpha \approx 7.79$ . This indicates

that for this task, MOEs scale poorly with parameter count, but are much more sensitive to shifts in parameter count compared to dense models. Fitting on activated parameters yields similar trends.

Finally, I attribute differences in  $E$  to noise from minimizing the Huber loss.

### 3 Fused Pallas Kernel

#### 3.1 Kernel Design

The kernel tiles the output  $B \times D$  matrix using tile sizes  $B_{\text{tile}}$  and  $D_{\text{tile}}$ , with an additional  $F_{\text{tile}}$  used for accumulation along the reduction dimension  $F$ . The kernel imitates `ragged_dot`'s strategy for handling ragged group boundaries; i.e. it calculates the starting and ending indices for  $B + E - 1$  logical tiles, such that each logical tile only spans the rows for a single expert (rows are padded to  $B_{\text{tile}}$ ). The kernel iterates over a  $D_{\text{tiles}} \times B_{\text{tiles}} \times F_{\text{tiles}}$  grid, indexed as  $(j, i, k)$ . At each grid step, the kernel loads the  $i$ -th  $B_{\text{tile}} \times D$  logical block from the input, the  $k$ -th  $D \times F_{\text{tile}}$  column block from  $W_{\text{up}}$ , and the  $j$ -th  $F_{\text{tile}} \times D_{\text{tile}}$  tile from  $W_{\text{down}}$ .

The iteration order is chosen to exploit how Pallas TPU kernels interact with memory references. For a fixed  $(j, i)$ , the output reference is fixed across all  $k$ , which allows safe accumulation into it. Because the operation is ragged, a single physical output tile can be split across multiple logical tiles at expert boundaries, so the kernel may need to revisit the same physical output reference more than once. Making  $j$  the slowest-changing grid dimension ensures these revisits happen consecutively, which allows it to accumulate safely.

#### 3.2 Experimental Results

I conducted 3 experiments to evaluate the isolated performance of the fused kernel relative to the unfused (i.e. imitating the computation in a single linear forward pass). In each plot, the  $y$ -axis shows the ratio of fused to unfused execution time (lower values indicate the fused kernel is faster).

### 3.3 Effect of Operand Dimension Size

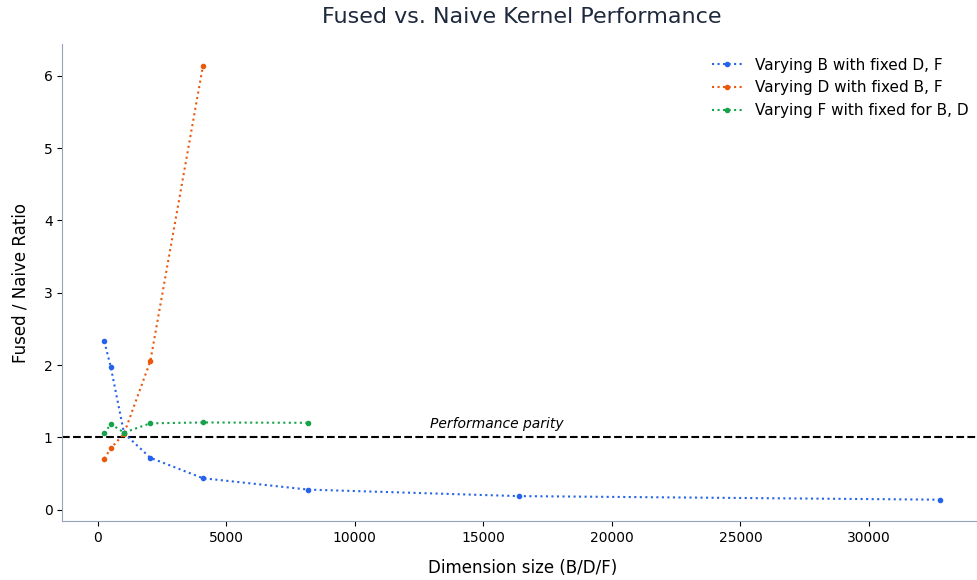


Figure 3: The  $x$ -axis plots the dimension size for each of  $B$ ,  $D$ , and  $F$ . Each curve displays how varying the dimension of each parameter influences performance.  $D$  and  $F$  are not fully plotted along the  $x$ -axis due to VMEM constraints. For this experiment, tile sizes for all parameters were fixed.

### 3.4 Effect of Tile Size

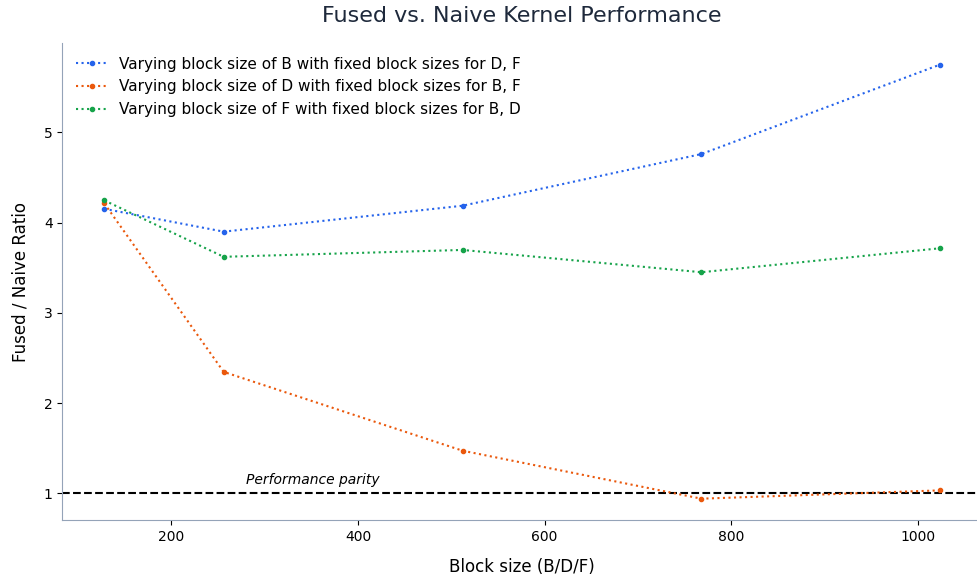


Figure 4: The  $x$ -axis plots the tile size for each of  $B$ ,  $D$ , and  $F$ . Each curve displays how varying the tile sizes of each parameter influences performance. For this experiment, the dimensions for all parameters were fixed.

### 3.5 Effect of Number of Experts

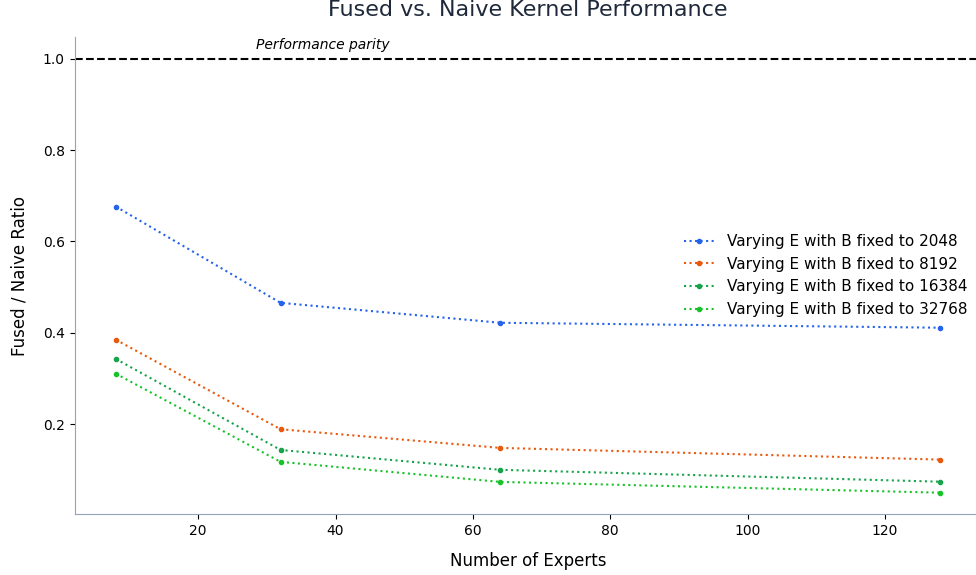


Figure 5: The  $x$ -axis plots the number of experts  $E$ . Each curve displays how varying  $E$  for a specific  $B$  influences performance. For this experiment,  $D$ ,  $F$ , and all tile sizes are fixed.

### 3.6 Takeaways

- Decreasing the number of times the fused kernel tiles over  $D$  drastically improves the fused vs. unfused ratio. I hypothesize this occurs because for each tile of  $D$  (indexed by  $j$ ), the kernel iterates over all  $k$  to accumulate into the output tile, and consequently reloads the entire  $W_{up}$  matrix once per  $j$ .
- Increasing  $B$  improves the fused vs. unfused ratio. This makes sense because the fused kernel avoids materializing the intermediate  $B \times F$  projection.
- Changing the tile sizes of  $B$  and  $F$  has minimal influence on the fused vs. unfused ratio. I hypothesize this occurs because varying the number of  $F$ -tiles does not alter total memory traffic, and because for sufficiently large  $B_{tile}$  the kernel becomes compute-bound, so that increasing  $B_{tile}$  further does not substantially improve performance. Notably, if  $B_{tile}$  is small enough that a single block contains rows routed to many different experts, the fused kernel’s performance degrades.
- Increasing the number of experts  $E$  improves the fused vs. unfused ratio. This result was unexpected: profiling revealed that the time taken for a single unfused up/down projection exceeded the total time taken by the full fused operation. This suggests that `ragged_dot` incurs a per expert overhead that the fused kernel avoids.

### 3.7 Transformer-Scale Forward Pass Comparison

Based on the analysis above, I made two transformer configurations for which I expected the fused kernel to significantly outperform the unfused baseline in the forward pass. Both configurations

use a batch size of 32768 and a sequence length of 16, giving an effective batch size of  $32768 \times 16 = 524288$ .

Table 2: Forward pass comparison between fused and unfused across two transformer configurations. Effective batch size is 524288 for both.

| Config | $D$ | $F$  | $H$ | $N$ | $K$ | $L$ | $E$ | $k$ | $tile_{b,d,f}$  | Unfused                  | Fused                    | Speedup       |
|--------|-----|------|-----|-----|-----|-----|-----|-----|-----------------|--------------------------|--------------------------|---------------|
| 1      | 256 | 1024 | 64  | 8   | 2   | 4   | 128 | 1   | (256, 256, 256) | 1.85 s $\pm$ 172 $\mu$ s | 316 ms $\pm$ 131 $\mu$ s | 5.85 $\times$ |
| 2      | 512 | 2048 | 32  | 4   | 2   | 1   | 128 | 1   | (256, 512, 256) | 1.6 s $\pm$ 2.62 ms      | 105 ms $\pm$ 217 $\mu$ s | 15.2 $\times$ |

Each timing is reported as mean  $\pm$  standard deviation over 100 runs.